

IC4302 - Proyecto 01: Semantify Book Reviews

Curso: Bases de Datos II (IC4302)

Semestre: Segundo Semestre 2025

Institución: Tecnológico de Costa Rica – Escuela de Ingeniería en Computación

VIDEO INFORMATIVO

[Que es PROMPTSY?](#)

Instrucciones de Ejecución

► Desplegar información

1.1 Requisitos Previos

- Cuenta en Docker Hub: Es un sitio web donde puedes guardar y compartir imágenes de programas listos para usar. Es como una "nube" para aplicaciones.
- Docker y Docker Compose: Docker es una herramienta que permite ejecutar programas en "contenedores", que son como cajas que traen todo lo necesario para que el programa funcione igual en cualquier computadora. Docker Compose ayuda a iniciar varios de estos programas juntos fácilmente.
- Kubernetes (Minikube o Docker Desktop): Kubernetes es una plataforma que ayuda a administrar y ejecutar muchos contenedores a la vez, ideal para proyectos grandes. Minikube y Docker Desktop son formas sencillas de usar Kubernetes en tu propia computadora.
- Helm Charts instalados: Helm es una herramienta que facilita la instalación y actualización de aplicaciones en Kubernetes, usando "charts" que son como recetas pre-hechas.
- Git: Es una herramienta para guardar y controlar los cambios en el código de un proyecto, permitiendo trabajar en equipo y mantener un historial de versiones.
- Lens: Es un programa con interfaz gráfica que permite ver y administrar fácilmente los recursos y servicios que se están ejecutando en Kubernetes.

1.2 Instalación de Componentes

1. Descargue el repositorio del proyecto en su computadora

```
git clone <URL_REPO>
```

Después ingrese a la carpeta del repositorio por medio de la terminal bash:

```
cd 2025-02-IC4302
```

2. Construya la imagenes de docker

Para poder realizar la construcción de las imágenes Docker. debe ingresar a la carpeta **docker** desde una terminal Bash y ejecutar el siguiente comando:

```
./build.sh usuario
```

NOTA: Sustituya la palabra usuario con su usuario de Docker Hub

3. Configure el registro de las imágenes para el chart

En su proyecto, ingrese a la carpeta de charts --> app --> templates --> values.yaml y registre el nombre de usuario en docker hub que desea utilizar en el campo **docker_registry**

```
config:
  docker_registry: SU_USUARIO # docker registry replace with your own username
  producer:
    enabled: true
```

3.2 Configuración de utilización de la aplicación web

Para la utilización de la aplicación web, se presentan dos opciones utilizar con Memcached o si este, para poder configurar su uso debe dirigirse a charts --> application-web --> values.yaml. Si usted desea utilizar el backend con Memcached debe habilitar la opción **useMemcached** en true, de lo contrario puede utilizarlo en false y se contruirá sin Memcached.

[!IMPORTANT]

La duración de construcción de la imagen HuggingFace puede tardar unos minutos.

```
config:
  docker_registry: SU_USUARIO
  backend:
    enabled: true
    replicas: 1
    name: backend-ui
    useMemcached: false #Variable para habilitar o deshabilitar Memcached
    image: backend-ui
    imageMemcached: backend-ui-memcached
  frontend:
    enabled: true
    replicas: 2
    name: frontend #this is the service name of frontend which represents the web
    application and the user interface
    image: frontend-ui
```

4. Instale el Helm Chart del proyecto

En su proyecto, ingrese a la carpeta de **charts** desde una terminal Bash y ejecute el siguiente comando:

```
./install.sh
```

5. Desinstalación del Helm Chart del proyecto

En caso de que usted necesite hacer la desinstalación del helm chart, ingrese a la carpeta de **charts** desde una terminal Bash y ejecute el siguiente comando:

```
./uninstall.sh
```

NOTA: Si no necesita la instalación, ignore este paso

6. Como ingresar a la pagina WEB PROMPTSY

► Desplegar información

Manual de acceso a la página web PROMPTSY

1. Asegúrese de que todos los servicios del proyecto estén desplegados y en ejecución (ver pasos anteriores de instalación).
2. Abra la aplicación **Lens** y seleccione la opción **Network** en el menú principal.
3. Una vez dentro, busque la opción de **services** y posteriormente busque el servicio llamado **frontend**
4. Ubique la parte donde indica los puertos y utilice el botón de **Forward**
5. Se mostrará la pantalla de inicio de sesión o registro de PROMPTSY.
 - Si ya tiene una cuenta, ingrese su correo y contraseña y presione "Ingresar".
 - Si no tiene cuenta, haga clic en "Register" o "Crear cuenta" y complete el formulario con sus datos (nombre, apellido, descripción, correo y contraseña).
6. Una vez autenticado, podrá navegar por todas las funcionalidades de la aplicación:
 - Buscar libros (Find Book)
 - Buscar y publicar prompts
 - Buscar amigos y gestionar su red
 - Ver y editar su perfil
 - Acceder al feed de actividad

Nota: Si la página no carga, verifique que el frontend esté desplegado y que no haya errores en los pods o servicios de Kubernetes/Docker.

Pruebas Unitarias

► Desplegar información

HuggingFace

► Desplegar información

Se realizaron tres pruebas unitarias con pytest a esta API para comprobar su funcionalidad:

- Verificación de que el endpoint encode retorne la información correcta
- Verificación de que el endpoint status retorne la información correcta
- Verificación de que se valide el campo "text"

 Imagen test 2025-10-08 a las 20 13 22_07747d3f

Crawler

► Desplegar información

Se realizaron pruebas unitarias para verificar el correcto funcionamiento del componente Crawler, encargado de la conexión con S3 y RabbitMQ, así como la configuración de métricas y logs. Estas pruebas incluyeron:

- Inicialización de métricas y servidor Prometheus: se validó que el módulo registre correctamente los contadores e histogramas de Prometheus, y que el servidor de métricas se inicie en el puerto 8000.
- Configuración de logging: se comprobó que el sistema de logging se inicialice correctamente al cargar el módulo.
- Conexión exitosa con RabbitMQ: se verificó la creación de la conexión, canales y colas (csv y parquet), junto con la autenticación mediante credenciales configuradas en las variables de entorno.
- Manejo de errores de conexión: se probó que, ante fallos en la conexión a RabbitMQ, se capture la excepción y se registre el error correspondiente.
- Publicación de mensajes exitosa: se comprobó que los mensajes se publiquen correctamente en la cola indicada, con el formato esperado.
- Manejo de errores en la publicación: se validó que, si ocurre un error al publicar, el sistema registre el mensaje de error sin interrumpir la ejecución.

 Imagen test Crawler

Ingest

► Desplegar información

Ingest CSV

► Desplegar información

Se realizaron pruebas unitarias para verificar el correcto funcionamiento del procesamiento de archivos CSV. Estas pruebas incluyeron:

- Validación de la existencia de los objetos en el almacenamiento (S3).

- Descarga y procesamiento de los archivos CSV.
- Formateo y normalización de los datos (fechas, ratings, texto).
- Inserción de información en la base de datos y relaciones entre libros, autores y categorías.
- Generación de embeddings y almacenamiento en Elasticsearch.

 Imagen test CSV

Ingest Parquet

► Desplegar información

Se realizaron pruebas unitarias para asegurar el correcto procesamiento de archivos Parquet. Estas pruebas incluyeron:

- Verificación de la existencia de los objetos en el almacenamiento (S3).
- Descarga y procesamiento de los archivos Parquet.
- Normalización de datos específicos de reviews (score, time, price).
- Inserción de información de reviews y libros en la base de datos.
- Generación de embeddings y almacenamiento en Elasticsearch.

 Imagen test Parquet

Backend UI Memcached

► Desplegar información

Se realizaron pruebas unitarias para verificar el correcto funcionamiento del componente Backend UI, encargado de la inicialización de la aplicación Flask, el registro de los blueprints y la respuesta del endpoint de salud (/health). Estas pruebas incluyeron:

- Registro de blueprints: se validó que se registren correctamente los cuatro blueprints principales del sistema (authentication, friend, prompt, user), utilizando los prefijos esperados: /promptsy/auth /promptsy/friend /promptsy/prompt /promptsy/user
- Verificación de inicialización de la aplicación: se comprobó que la aplicación Flask cargue correctamente los módulos de rutas y ejecute el proceso de registro sin errores.
- Endpoint de salud: se verificó que el endpoint /health responda con un código de estado 200 OK y un cuerpo JSON con el contenido {"status": "ok"}.
- Aislamiento mediante mocks: se emplearon monkeypatch y MagicMock para simular las dependencias externas y garantizar que las pruebas se ejecuten sin necesidad de instancias reales.

 Imagen test Backend Ui Memcached

Backend UI

► Desplegar información

Se realizaron pruebas unitarias para verificar el correcto funcionamiento del componente Backend UI sin Memcached, el cual gestiona la inicialización de la aplicación Flask, el registro de los blueprints principales y la respuesta del endpoint /health. Estas pruebas incluyeron:

- Registro de blueprints: se comprobó que la aplicación registre los cuatro blueprints principales del sistema (authentication, friend, prompt, user), asegurando que el método register_blueprint se invoque exactamente cuatro veces y que todos los módulos sean correctamente detectados.
- Validación de la aplicación Flask: se verificó que la aplicación Flask se inicialice de forma correcta, cargando los módulos de rutas simulados mediante monkeypatch sin requerir dependencias reales.
- Endpoint de salud: se validó que el endpoint /health devuelva una respuesta con código 200 OK y el cuerpo JSON {"status": "ok"}, confirmando que el servicio se encuentra activo.
- Uso de mocks controlados: se implementaron MagicMock y monkeypatch para aislar las dependencias externas y asegurar que las pruebas se ejecuten de manera controlada y reproducible.

 Imagen test Backend Ui Memcached

Configuración de componenetes

► Details

Desplegar información

S3 Crawler

► Desplegar información

El S3 Crawler es un Cron Job que se ejecuta cada hora, recorre el bucket con una lista de prefijos y lista los objetos .json y .parquet, publicándolos en RabbitMQ. Tiene el siguiente flujo:

 Flow Chart Crawler

- **RabbitMQ:** para ver los mensajes publicados en RabbitMQ, se puede abrir el puerto 15672, con el siguiente comando:

```
kubect1 port-forward svc/databases-rabbitmq 15672:15672 -n default
```

Después, se ingresa al enlace "<http://localhost:15672/>", y con su usuario y contraseña puede acceder a ver las colas y mensajes.

Ingest

► Desplegar información

El ingest se divide en dos componentes, el Ingest CSV y el Ingest Parket. El Ingest CSV se encarga de procesar los books. Estos vienen en la cola de rabbitmq como una key de un AWS bucket, en formato .json. El Ingest CSV se encarga de descargar los archivos, guardarlos en una lista de diccionarios, conseguir los embeddings, subir los datos a elasticsearch y a mariadb. El ingest Parket funciona de la misma manera, solo que hace el mismo proceso con las reviews que vienen en formato .parquet.

Implementación de MariaDB

► Desplegar información

Se ha implementado la base de datos **promptsy** en MariaDB, la cual utiliza dos esquemas principales para organizar la información del proyecto:

- **Esquema 1:** Gestión de libros, autores, categorías y reseñas (reviews). Este esquema permite almacenar toda la información relacionada con los libros, sus autores, categorías y las reseñas asociadas. Incluye tablas como **books**, **authors**, **categories**, **reviews**, y tablas intermedias para relaciones.
- **Esquema 2:** Gestión de usuarios, prompts, likes, amigos y relaciones sociales. Este esquema está enfocado en la funcionalidad social de la aplicación, permitiendo registrar usuarios, sus prompts, likes, amistades y relaciones entre ellos. Incluye tablas como **User**, **Prompt**, **Liked**, **Friend**.

La siguiente imagen muestran la estructura del esquema de promptsy (red social):

Esquema 2

 Diagramas P1-Bases de datos- Promptsy

Cada esquema está diseñado para facilitar la integración entre la gestión de contenido (libros y reseñas) y la interacción social (usuarios y prompts), permitiendo consultas eficientes y una experiencia completa en la aplicación.

Inicialización de Elasticsearch

► Desplegar información

El despliegue de Elasticsearch se realiza mediante un Job que inicializa los índices requeridos para la aplicación. Este job se encuentra en el directorio **P1/Docker/Elastic-Init** y utiliza el archivo **elastic-mappings.json** para definir el mapping de cada índice.

Índices y su propósito

- **books:** Almacena información de libros y su vector de embeddings. Se utiliza para búsquedas vectoriales (vector search) mediante el campo **embeddings**.
- **reviews:** Almacena reseñas de libros y su vector de embeddings. También se utiliza para búsquedas vectoriales (vector search) sobre reseñas.
- **nbooks:** Almacena información de libros pero solo para búsquedas de texto (text search), sin campo de embeddings indexado.
- **nreviews:** Almacena reseñas de libros para búsquedas de texto (text search), sin campo de embeddings indexado.

Mapping de los índices

```
// books
"books": {
  "mappings": {
    "properties": {
      "title": {"type": "text"},
      "description": {"type": "text"},
      "authors": {"type": "text"},
```

```

    "categories": {"type": "text"},
    "publisheddate": {"type": "date", "ignore_malformed": true},
    "publisher": {"type": "text"},
    "previewlink": {"type": "text"},
    "infolink": {"type": "text"},
    "image": {"type": "text"},
    "ratingscount": {"type": "float", "ignore_malformed": true},
    "embeddings": {"type": "dense_vector", "dims": 768, "index": true,
"similarity": "cosine"}
  }
}
},
// reviews
"reviews": {
  "mappings": {
    "properties": {
      "title": {"type": "text"},
      "price": {"type": "float", "ignore_malformed": true},
      "user_id": {"type": "keyword"},
      "profilename": {"type": "text"},
      "review/helpfulness": {"type": "keyword"},
      "review/score": {"type": "float", "ignore_malformed": true},
      "review/time": {"type": "date", "ignore_malformed": true},
      "review/summary": {"type": "text"},
      "review/text": {"type": "text"},
      "book_id": {"type": "keyword"},
      "embeddings": {"type": "dense_vector", "dims": 768, "index": true,
"similarity": "cosine"}
    }
  }
},
// nbooks
"nbooks": {
  "mappings": {
    "properties": {
      "title": {"type": "text"},
      "description": {"type": "text"},
      "authors": {"type": "text"},
      "categories": {"type": "text"},
      "publisheddate": {"type": "date", "ignore_malformed": true},
      "publisher": {"type": "text"},
      "previewlink": {"type": "text"},
      "infolink": {"type": "text"},
      "image": {"type": "text"},
      "ratingscount": {"type": "float", "ignore_malformed": true}
    }
  }
},
// nreviews
"nreviews": {
  "mappings": {
    "properties": {
      "title": {"type": "text"},
      "price": {"type": "float", "ignore_malformed": true},

```



```

    "user_id": {"type": "keyword"},
    "profilename": {"type": "text"},
    "review/helpfulness": {"type": "keyword"},
    "review/score": {"type": "float", "ignore_malformed": true},
    "review/time": {"type": "date", "ignore_malformed": true},
    "review/summary": {"type": "text"},
    "review/text": {"type": "text"},
    "book_id": {"type": "keyword"}
  }
}
}

```

Notas:

- Los índices **books** y **reviews** permiten búsquedas vectoriales gracias al campo **embeddings** indexado y con similitud **cosine**.
- Los índices **nbooks** y **nreviews** están optimizados para búsquedas de texto tradicional y no incluyen el campo de embeddings indexado.

Esto permite a la aplicación realizar tanto búsquedas semánticas (vector search) como búsquedas clásicas de texto (text search) de manera eficiente.

UI

► Desplegar información

Uso de IA para la UI

► Desplegar información

Para el proyecto 01 se utiliza Inteligencia Artificial para la elaboración de la interfaz de Usuarios, específicamente para hacer el diseño de todas las pantallas que aquí se describen, para esto se utilizó V0, el asistente de IA de Vercel la cual está enfocada en desarrollo web y aplicaciones full-stack, está construida sobre Claude de Anthropic, específicamente Claude 3.5 Sonnet.

Se hace un prompt específico para cada una de las pantallas brindando detalles de cómo se quiere que luzca, a continuación, los prompts suministrados

1. Login y Register

Objetivo: Generar el módulo Register y login para mi aplicación React (Vite).

Estructura esperada:

- o /Login/Login.tsx
- o /Login/Login.module.css

Requisitos técnicos:

- o Usar React con Javascript (.jsx).
- o Importar estilos con CSS Modules (import s from './register.module.css';).

UI esperada:

- o Card centrada con título "registro".
- o Inputs para correo y contraseña, con labels.link "¿Olvidaste tu contraseña?".
- o Botón principal "Ingresar".

- o Estado de error: si envío vacío, mostrar mensaje en rojo o las correspondientes para register como name lastname description email y password
- o Login/Register: formularios con validaciones mínimas; enlaces entre ellos.

Output esperado:

- o El código completo de Login.jsx y Register.jsx
- o El código completo de Login.module.css y Register.module.css.
- o Todo debe ser en inglés.

Importante: Devuélveme únicamente esos dos archivos.

2. Find Book

Objetivo: Generar el módulo Find book para mi aplicación React (Vite).

Estructura esperada:

- o /Ask/ Ask.jsx
- o / Ask / Ask.module.css

Requisitos técnicos:

- o Usar React con Javascript (.jsx).
- o Importar estilos con CSS Modules (import s from "./ask.module.css";).

UI esperada:

- o Textarea de prompt + botón “Buscar”; renderiza resultados de 5 fuentes (placeholders):vector search, 2) vector reviews, 3) text books, 4)text reviews 5)mariadb Cada card muestra título, autores, descripción, published date, previewlink, publisher, published date, más información, rating, categorías, etc.
- o Enviar prompt al Feed con botón “Publicar en mi Feed”.
- o Cada una de las páginas principales (esta es una principal) tendrá una barra inferior con las opciones a las que se pueden ir, find book, friends, prompts, feed y me. Idea similar a la barra que tiene instagram con las opciones.
- o Dos temas disponibles: colorido (gradientes suaves, acentos índigo/morado/naranja) y formal (azul oscuro/grises/blanco).
- o Estilo moderno, minimalista, accesible (labels, aria-*), sombras suaves, radios xl/2xl.

Output esperado:

- o El código completo de Ask.jsx
- o El código completo de Ask.module.css
- o Todo debe ser en inglés.

Importante: Devuélveme únicamente esos dos archivos.

3. Search Prompts

Objetivo: Generar el módulo Search Prompts para mi aplicación React (Vite).

Estructura esperada:

- o /Prompt/ prompt.jsx
- o / Prompt / prompt.module.css

Requisitos técnicos:

- o Usar React con Javascript (.jsx).
- o Importar estilos con CSS Modules (import s from "./prompt.module.css";).

UI esperada:

- o Una búsqueda de prompts que permita buscar por prompt o por nombre de usuario con un input de búsqueda

- o Debe salir listado todos los prompts con su debido texto y el nombre del usuario.
- o A cada prompt que salga tenga la posibilidad de darle like y el count de likes también debe estar.
- o Cada una de las páginas principales (esta es una principal) tendrá una barra inferior con las opciones a las que se pueden ir, find book, friends, prompts, feed y me. Idea similar a la barra que tiene instagram con las opciones.
- o Dos temas disponibles: colorido (gradientes suaves, acentos índigo/morado/naranja) y formal (azul oscuro/grises/blanco).
- o Estilo moderno, minimalista, accesible (labels, aria-*), sombras suaves, radios xl/2xl.

Output esperado:

- o El código completo de Prompt.jsx
- o El código completo de Prompt.module.css
- o Todo debe ser en inglés.

Importante: Devuélveme únicamente esos dos archivos.

4. Search Friends

Objetivo: Generar el módulo Search Friends para mi aplicación React (Vite).

Estructura esperada:

- o /Friends/ friends.jsx
- o / Friends / friends.module.css

Requisitos técnicos:

- o Usar React con Javascript (.jsx).
- o Importar estilos con CSS Modules (import s from "./friends.module.css";).

UI esperada:

- o Una búsqueda de friends que permita buscar por nombre de usuario con un input de búsqueda.
- o Debe salir listado todas las coincidencias con el nombre del usuarios, que cada usuario que salga tenga la posibilidad de darle follow y unfollow.
- o Cada una de las páginas principales (esta es una principal) tendrá una barra inferior con las opciones a las que se pueden ir, find book, friends, prompts, feed y me. Idea similar a la barra que tiene instagram con las opciones.
- o Dos temas disponibles: colorido (gradientes suaves, acentos índigo/morado/naranja) y formal (azul oscuro/grises/blanco).
- o Estilo moderno, minimalista, accesible (labels, aria-*), sombras suaves, radios xl/2xl.

Output esperado:

- o El código completo de Friends.jsx
- o El código completo de Friends.module.css
- o Todo debe ser en inglés.

Importante: Devuélveme únicamente esos dos archivos.

5. Feed

Objetivo: Generar el módulo Feed para mi aplicación React (Vite).

Estructura esperada:

- o /Feed/ feed.jsx

- o / Feed / feed.module.css

Requisitos técnicos:

- o Usar React con Javascript (.jsx).
- o Importar estilos con CSS Modules (import s from "./feed.module.css";).

UI esperada:

- o Feed: cards cronológicas con este diseño:
 - Avatar del usuario (círculo pequeño).
 - Nombre en bold.
 - Botón Follow/Unfollow al lado del nombre, similar a Instagram.
 - Debajo, el texto del prompt.
 - Al pie, las métricas de likes con un ícono  y el número. La idea es que sea similar a un feed de instagram donde las fotos son más bien prompts.
- o Cada una de las páginas principales (esta es una principal) tendrá una barra inferior con las opciones a las que se pueden ir, find book, friends, prompts, feed y me. Idea similar a la barra que tiene instagram con las opciones.
- o Dos temas disponibles: colorido (gradientes suaves, acentos indigo/morado/naranja) y formal (azul oscuro/grises/blanco).
- o Estilo moderno, minimalista, accesible (labels, aria-*), sombras suaves, radios xl/2xl.

Output esperado:

- o El código completo de Feed.jsx
- o El código completo de Feed.module.css
- o Todo debe ser en inglés.

Importante: Devuélveme únicamente esos dos archivos.

6. Me

Objetivo: Generar el módulo Me para mi aplicación React (Vite).

Estructura esperada:

- o /Me/ me.jsx
- o / Me / me.module.css

Requisitos técnicos:

- o Usar React con Javascript (.jsx).
- o Importar estilos con CSS Modules (import s from "./me.module.css";).

UI esperada:

- o Me: formulario para editar perfil.
- o Tabla/lista de prompts propios con Edit/Delete.
- o Botón de logout.
- o Count de followers y following con cada ususario.
- o Cada una de las páginas principales (esta es una principal) tendrá una barra inferior con las opciones a las que se pueden ir, find book, friends, prompts, feed y me. Idea similar a la barra que tiene instagram con las opciones.
- o Dos temas disponibles: colorido (gradientes suaves, acentos indigo/morado/naranja) y formal (azul oscuro/grises/blanco).
- o Estilo moderno, minimalista, accesible (labels, aria-*), sombras suaves, radios xl/2xl.

Output esperado:

- o El código completo de Me.jsx
- o El código completo de Me.module.css
- o Todo debe ser en inglés.

Importante: Devuélveme únicamente esos dos archivos.

7. My Friends

Objetivo: Generar el módulo My friends para mi aplicación React (Vite).

Estructura esperada:

- o /MyFriends/ MyFriends.jsx
- o / MyFriends / MyFriends.module.css

Requisitos técnicos:

- o Usar React con Javascript (.jsx).
- o Importar estilos con CSS Modules (import s from "./myFriends.module.css";).

UI esperada:

- o Cards con nombre en bold, avatar del usuario y botón para follow y unfollow.

Diseño similar a cuando se despliegan los seguidos en Instagram.

- o Cada una de las páginas principales (esta es una principal) tendrá una barra inferior con las opciones a las que se pueden ir, find book, friends, prompts, feed y me. Idea similar a la barra que tiene instagram con las opciones.

- o Dos temas disponibles: colorido (gradientes suaves, acentos índigo/morado/naranja) y formal (azul oscuro/grises/blanco).

- o Estilo moderno, minimalista, accesible (labels, aria-*), sombras suaves, radios xl/2xl.

Output esperado:

- o El código completo de MyFriends.jsx
- o El código completo de MyFriends.module.css
- o Todo debe ser en inglés.

Importante: Devuélveme únicamente esos dos archivos.

Registro / Inicio de sesión

Se implementan dos pantallas, una para crear cuenta y otra para iniciar sesión. En el registro se validan los campos básicos nombre completo, correo, descripción y contraseña. Toda la información relacionada al cliente es guardado en MariaDB. En el inicio de sesión se toma el correo y la contraseña para la validación de existencia del usuario.



Menú de navegación

Al entrar, la persona podrá ver una barra de navegación para poder ingresar a cualquiera de las distintas opciones y navegar fácilmente a cada función

- Find Book
- Search Prompts
- Find Friends
- Feed
- Friends
- Me

Find Book

Para Find Book se presenta un campo de búsqueda, donde el usuario escribe la necesidad de tipo de libros que desea buscar y con respecto a su petición podrá tener dos opciones: **Buscar** o **Publicar**. Al buscar podrá obtener resultados de vector search y reviews, text books y reviews y por último de MariaDB, en los que podrá consultar información general de los libros obtenidos.

Al utilizar la opción de publicar, el prompt o petición del usuario será publicado en el feed para ser consultado en un futuro o para que sea visto por sus amigos.



Search Prompts

En Search Prompts se realiza una barra de búsqueda que filtra prompts por el texto y por el nombre de usuario. Cada tarjeta de prompt mostrada incluye la acción de "like". Cuando el usuario da like, se realiza una actualización de likes en los prompts del usuario y ese prompt también se agrega al Feed del usuario que dio like.



Find Friends

Para Find Friends se construye un buscador por nombre. Cada resultado se presenta con un pequeño perfil con el nombre del usuario y cantidad de followers, además de un botón de "Follow" para poder agregarlo a sus amigos.



Feed

El Feed mezcla los prompts propios y los de la gente que el usuario sigue, ordenados del más reciente al más antiguo. También al lado del prompt puede encontrar el botón de **buscar** de esta manera si el usuario quiere buscar los resultados que puede arrojar ese prompt lo puede hacer.



Friends

La sección Friends lista a las personas que el usuario ya sigue, así como la opción de dejarlos de seguir si así lo decide.



Me

En Me se centralizan dos cosas: el perfil del usuario y la gestión de sus prompts. Para el perfil se muestra el nombre, apellido, descripción y correo. Se permite editarlos.

Para los prompts propios, se muestra una lista editable con opciones para modificar o borrar el prompt. La idea con esta funcionalidad es que el usuario tenga control total de su información y contenido de manera fácil.

Por último se presenta el botón de **logout** para cerrar sesión y redirigir a la pantalla de inicio de sesión.



API

► Desplegar información

Arquitectura General

El API REST del backend está construido con Flask y expone endpoints organizados en 4 módulos principales:

- **Authentication:** Gestión de registro e inicio de sesión
- **User:** Gestión de información de usuario
- **Prompt:** Gestión de prompts y búsquedas
- **Friend:** Gestión de relaciones sociales y likes

Todos los endpoints utilizan conexión a MariaDB o ElasticSearch mediante connection pooling y bcrypt para el manejo seguro de contraseñas.

► Ver Endpoints

Authentication Endpoints

Base URL: `/promptsy/auth`

1. Login

```
POST /promptsy/auth/login
```

Descripción: Autentica a un usuario mediante email y contraseña.

Request Body:

```
{
  "email": "string",
  "password": "string"
}
```

Response Success (200):

```
{
  "id": "integer",
  "name": "string",
  "lastname": "string",
  "description": "string",
  "email": "string"
}
```

Errores:

- **400:** Email y password son requeridos

- 401: Credenciales inválidas
 - 500: Error de base de datos
-

2. Register

```
POST /promptsy/auth/register
```

Descripción: Registra un nuevo usuario en el sistema.

Request Body:

```
{
  "name": "string",
  "lastname": "string",
  "description": "string",
  "email": "string",
  "password": "string"
}
```

Response Success (201):

```
{
  "message": "User registered successfully"
}
```

Errores:

- 400: Campos requeridos faltantes o email ya existe
- 500: Error de base de datos

Nota: La contraseña se hashea con bcrypt antes de almacenarse.

User Endpoints

Base URL: /promptsy/user

1. Get User Information

```
GET /promptsy/user/me?id={user_id}
```

Descripción: Obtiene la información completa de un usuario.

Response Success (200):

```
{
  "id": "integer",
  "name": "string",
  "lastname": "string",
  "description": "string",
  "email": "string"
}
```

2. Edit User Information

```
PUT /promptsy/user/edit
```

Descripción: Actualiza nombre, apellido y descripción del usuario.

Request Body:

```
{
  "id": "integer",
  "name": "string",
  "lastname": "string",
  "description": "string"
}
```

Response Success (201):

```
{
  "message": "User edited successfully"
}
```

3. Change Password

```
POST /promptsy/user/change-password
```

Descripción: Cambia la contraseña del usuario validando la contraseña anterior.

Request Body:

```
{
  "id": "integer",
  "oldpass": "string",
  "newpass": "string"
}
```

Response Success (201):

```
{
  "message": "Password edited successfully"
}
```

Errores:

- **401:** Contraseña antigua incorrecta
-

Prompt Endpoints**Base URL:** /promptsy/prompt**1. Generate prompt**

```
POST /promptsy/prompt/generate
```

Descripción: Genera resultados de búsquedas en cada tipo de búsqueda ofrecida.**Request Body:**

```
{
  "text": "string"
}
```

Nota: Endpoint en desarrollo para búsqueda semántica.

2. Post Prompt

```
POST /promptsy/prompt/post
```

Descripción: Crea un nuevo prompt.**Request Body:**

```
{
  "prompt": {
    "text": "string",
    "id_user": "integer"
  }
}
```

Response Success (201):

```
{
  "message": "Prompt registered successfully"
}
```

3. Edit Prompt

```
PUT /promptsy/prompt/edit
```

Descripción: Edita el texto de un prompt existente.

Request Body:

```
{
  "prompt": {
    "id_prompt": "integer",
    "text": "string"
  }
}
```

4. Delete Prompt

```
PUT /promptsy/prompt/delete
```

Descripción: Realiza un borrado lógico del prompt (enabled = FALSE).

Request Body:

```
{
  "id_prompt": "integer"
}
```

5. Get My Prompts

```
GET /promptsy/prompt/myprompts?id_user={user_id}
```

Descripción: Obtiene todos los prompts de un usuario.

Response Success (200):

```
[
  {
    "id": "integer",
    "text": "string",
    "created_at": "timestamp",
    "likes": "integer",
    "name": "string",
    "lastname": "string"
  }
]
```

6. Get Single Prompt

```
GET /promptsy/prompt/myprompt?id_prompt={prompt_id}
```

Descripción: Obtiene un prompt específico por su ID.

7. Search Prompts

```
GET /promptsy/prompt/search?text={search_text}
```

Descripción: Busca prompts por texto, nombre o apellido de usuario.

Response Success (200):

```
[
  {
    "id": "integer",
    "text": "string",
    "name": "string",
  }
]
```

```
    "lastname": "string",  
    "likes": "integer"  
  }  
]
```

Implementación:

- Divide el texto en palabras
 - Busca coincidencias con operador LIKE
 - Todas las palabras deben coincidir (AND)
-

8. Get Feed

```
GET /promptsy/prompt/feed?id_user={user_id}
```

Descripción: Obtiene el feed personalizado (prompts de amigos y prompts con like).

Response Success (200):

```
[  
  {  
    "id": "integer",  
    "text": "string",  
    "created_at": "timestamp",  
    "likes": "integer",  
    "name": "string",  
    "lastname": "string"  
  }  
]
```

Friend Endpoints

Base URL: /promptsy/friend

1. Follow User

```
POST /promptsy/friend/follow
```

Descripción: Permite seguir a otro usuario.

Request Body:

```
{
  "id_user": "integer",
  "id_friend": "integer"
}
```

Response Success (201):

```
{
  "message": "User followed successfully"
}
```

Validaciones:

- No puede seguirse a sí mismo
- No puede seguir al mismo usuario dos veces
- El usuario a seguir debe existir

Efectos:

- Incrementa contador **following** del usuario
 - Incrementa contador **followers** del usuario seguido
-

2. Unfollow User

```
PUT /promptsy/friend/unfollow
```

Descripción: Deja de seguir a un usuario.

Request Body:

```
{
  "id_user": "integer",
  "id_friend": "integer"
}
```

Efectos:

- Borrado lógico (enabled = FALSE)
 - Decrementa contadores de following/followers
-

3. Find Friends

```
GET /promptsy/friend/find?text={search_text}
```

Descripción: Busca usuarios por nombre o apellido.

Response Success (200):

```
[
  {
    "id": "integer",
    "name": "string",
    "lastname": "string",
    "email": "string"
  }
]
```

4. Get Friends

```
GET /promptsy/friend/get_friends?id={user_id}
```

Descripción: Obtiene la lista de usuarios que el usuario sigue.

Response Success (200):

```
[
  {
    "id": "integer",
    "name": "string",
    "lastname": "string",
    "description": "string",
    "email": "string",
    "followers": "integer",
    "following": "integer"
  }
]
```

5. Like Prompt

```
POST /promptsy/friend/like
```

Descripción: Da "like" a un prompt.

Request Body:

```
{
  "id_user": "integer",
  "id_prompt": "integer"
}
```

Efectos:

- Incrementa contador de likes del prompt
 - El prompt aparece en el feed del usuario
-

6. Unlike Prompt

```
PUT /promptsy/friend/unlike
```

Descripción: Quita el "like" de un prompt.

Request Body:

```
{
  "id_user": "integer",
  "id_prompt": "integer"
}
```

Efectos:

- Borrado lógico (enabled = FALSE)
 - Decrementa contador de likes del prompt
-

Características Técnicas**Seguridad**

- Contraseñas hasheadas con bcrypt
- Consultas parametrizadas (prevención de SQL injection)
- No se devuelven contraseñas en respuestas

Connection Pooling

- Conexiones reutilizables a MariaDB
- Optimización de recursos y rendimiento
- Implementado en `mariadb_connection.py`

- Conexiones reutilizables a Elasticsearch
- Implementado en `elastic_connection.py`

Borrado Lógico

- Campo `enabled` para soft delete
- Permite auditoría y recuperación de datos
- Aplicado en: Friend, Liked, Prompt

Logging

- Logs estructurados con nivel INFO/WARNING/ERROR
- Trazabilidad de operaciones
- Integración con sistemas de observabilidad

Versión con Memcached

Disponible en `BackendUI_Memcached` con:

- Cache de endpoints de lectura
- Métricas de cache hit/miss
- TTL configurable

Health Check

```
GET /health
```

Verifica disponibilidad del servicio para Kubernetes.

HuggingFace API

► Desplegar información

Esta API es la encargada de generar los embeddings, esto se logra por medio del modelo "sentence-transformers/all-mpnet-base-v2". Este modelo permite que obtenga el texto y lo transforma en vectores. La API está conformada de 3 endpoints:

1. Generate Embedding

```
POST/encode
```

Descripción: Genera un embedding a partir de un texto utilizando el modelo sentence-transformers/all-mpnet-base-v2

Request Body:

```
{
  "text": "string"
}
```

Response Body:

```
{
  "text": "string",
  "embedding": [0.123, -0.456, 0.789, ...]
}
```

2. Health Check

```
GET /status
```

Descripción: Verifica que la API esté funcionando correctamente

Response Body:

```
{
  "text": "string",
  "embedding": [0.123, -0.456, 0.789, ...]
}
```

3. Prometheus Metrics

```
GET /metrics
```

Descripción: Expone métricas de Prometheus para monitoreo de la API

Métricas Disponibles:

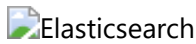
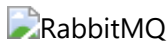
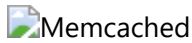
- total_peticiones_http
- promedio_tiempo_embedding

Observabilidad

► Desplegar información

 Api

 Crawler



Conclusiones

► Desplegar información

1. El uso de connection pooling en las bases de datos permite reutilizar conexiones existentes en lugar de crear una nueva cada vez que se requiere comunicación. Esto optimiza el uso de recursos, mejora el rendimiento del sistema y reduce la latencia en las operaciones.
2. La correcta utilización de los métodos HTTP resulta fundamental para definir de manera clara y estandarizada cómo se manipula la información que es enviada o recibida a través de los endpoints. Su adecuada implementación favorece la coherencia y la mantenibilidad servicios.
3. Utilizar React para desarrollo permite crear interfaces flexibles gracias a la gran cantidad de librerías que contiene y a su arquitectura basada en componnetes.
4. Al desarrollar la aplicación se establece una interfaz intuitiva para la interacción con el sistema, se crea una aplicación similar a una red social para que los usuarios puedan descubrir nuevos libros por medio de búsquedas vectoriales y a la vez compartir sus búsquedas con otras personas.
5. El uso de Memcached mejora significativamente el rendimiento de la API, ya que evita llamadas redundantes a la base de datos y optimiza la experiencia de usuario al entregar respuestas más rápidas.
6. El uso de logs mejora la capacidad de monitoreo y depuración de la aplicación, permitiendo identificar errores, clasificar el tipo y llevar un registro del tiempo de cada uno.
7. Utilizar el servicio AWS S3 como fuente de datos y RabbitMQ como cola intermedia permite un flujo eficiente en el flujo y en el procesamiento, permitiendo usar esta infraestructura para datasets grandes.
8. El uso de una arquitectura modular, donde el crawler y los consumidores funcionan como componentes independientes, facilita la escalabilidad y el mantenimiento del sistema.
9. La comunicación constante dentro del equipo permitió coordinar tareas de manera efectiva, logrando así la integración de las diferentes partes del sistema de manera exitosa, permitiendo una división de responsabilidades y un desarrollo eficiente.
10. La colaboración entre miembros del equipo, combinando distintas habilidades y perspectivas, resultó en soluciones para problemas que se presentaron durante la elaboración del proyecto.

Recomendaciones

► Desplegar información

1. **Uso de variables de entorno** Se recomienda centralizar la configuración del sistema mediante variables de entorno, lo cual facilita la mantenibilidad y portabilidad de la aplicación. Estas variables deben incluir, entre otros aspectos, las credenciales y parámetros de conexión a la base de datos, así como las direcciones y claves necesarias para el consumo de endpoints externos.
2. **Separación del esquema de base de datos y el código de aplicación** Se recomienda generar un script de inicialización de la base de datos, integrado en los Helm Charts, que permita separar la creación y configuración del esquema de la base de datos del código de la aplicación en producción. Esto asegura mayor control y trazabilidad en la gestión del ciclo de vida de la base de datos, evitando acoplamiento con el código de negocio. También facilita la automatización de despliegues y la aplicación de migraciones en entornos de desarrollo, pruebas y producción de manera ordenada y consistente.
3. **Al desarrollar la UI se recomienda separar la vista de los datos**, de manera que exista un componente que se encargue únicamente de hacer las peticiones, esto permite mejor entendimiento y facilidad a la hora de detectar errores.
4. **Utilizar mensajes para el usuario a la hora de que interactue con el sistema**, al momento de que algo sale bien o algo sale mal. Esto permite que tengan mejor entendimiento de lo que se está haciendo y mejora la experiencia de uso al evitar confusiones.
5. **Implementar Memcached para almacenar los resultados de consultas frecuentes** permite reducir la carga sobre la base de datos y minimizar la latencia en las respuestas. También, es recomendable definir un tiempo de expiración apropiado para los datos cacheados.
6. **Usar logs en lugar de prints como se hacía anteriormente**, ya que este es mejor debido que se puede definir por categorías los mensajes (info, warning, error, debug). Además, de que nos indican el timestamp lo cual es muy beneficioso y no lo realizan los prints.
7. **Se recomienda aprovechar los servicios cloud y los patrones de mensajería modernos**, como AWS S3 y como RabbitMQ, para así poder tener escalabilidad en los proyectos, estando preparados para trabajar con datasets enormes.
8. **Al trabajar en procesamiento de datos grandes**, se recomienda seguir el patrón producer-consumer. En el caso del proyecto esto se ve en la separación de componentes: Crawler, Ingest y una cola que los comunica, esto favorece la escalabilidad horizontal, el añadir cosas nuevas y la mantenibilidad.
9. **Fomentar reuniones periódicas dentro del equipo** para mantener una comunicación clara sobre el estado de las tareas, evitando malentendidos y retrasos en el proyecto.
10. **Promover un ambiente de colaboración** donde cada miembro del equipo pueda tomar una responsabilidad lo que fortaleciendo la cohesión y mejorando la calidad del trabajo colectivo.

Referencias

► Desplegar información

- <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>
- <https://www.ibm.com/think/topics/api-endpoint>
- <https://mariadb.com/docs/connectors/mariadb-connector-python/pooling>
- <https://discuss.elastic.co/t/connection-pooling-using-python-client-8-10/344763>
- <http://elastic.co/docs/reference/elasticsearch/clients/python/connecting>
- <https://www.elastic.co/what-is/vector-search>
- <https://www.elastic.co/docs/solutions/search/vector>
- https://www.w3schools.com/sql/sql_like.asp
- <https://realpython.com/api-integration-in-python/>
- <https://mariadb.com/docs/tools/mariadb-enterprise-operator/installation/helm>
- <https://react.dev/learn/managing-state>
- <https://docs.aws.amazon.com/AmazonS3/latest/userguide/directory-buckets-objects-GetExamples.html>
- <https://docs.python.org/3/library/json.html#json.loads>
- https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_parquet.html
- <https://elasticsearch-py.readthedocs.io/en/v8.11.1/helpers.html>
- <https://docs.python.org/3/library/datetime.html>
- <https://docs.python.org/3/library/datetime.html>
- <https://kinsta.com/blog/indexing-in-mysql-mariadb/>
- <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/dense-vector>
- <https://docs.pytest.org/en/stable/how-to/monkeypatch.html>
- <https://docs.python.org/3/library/threading.html>
- <https://prometheus.io/docs/practices/pushing/>
- https://prometheus.github.io/client_python/exporting/http/
- https://prometheus.github.io/client_python/exporting/pushgateway/

Tabla de Estado

► Desplegar información

Componente	Estado	Notas / Pendientes
------------	--------	--------------------

Componente	Estado	Notas / Pendientes
Hugging Face API	100%	
S3 Crawler Cron Job	100%	
Ingest	100%	
API	100%	
API con Memcached	100%	
UI (Frontend)	100%	
Base de Datos	100%	
Observabilidad de componentes	100%	
Componentes automatizados	100%	
Pruebas (unitarias/integración)	100%	